UCab – Cab Booking Platform

1. INTRODUCTION

1.1 Project Overview

UCab is a full-stack cab booking platform developed using the **MERN stack (MongoDB, Express.js, React.js, Node.js)**. The system allows passengers to book rides and drivers to accept ride requests through an online platform. It provides features such as user authentication, ride booking, driver availability management, trip tracking, and trip history.

The platform demonstrates how modern web technologies can be used to build scalable transportation services similar to ride-sharing applications. UCab focuses on providing a simple interface for users to request rides and for drivers to manage ride requests efficiently.

1.2 Purpose

The purpose of the UCab project is to design and develop a web-based cab booking system that connects passengers with available drivers. The system aims to simplify the process of requesting rides, managing trips, and maintaining ride history through a centralized digital platform.

2. IDEATION PHASE

2.1 Problem Statement

Traditional transportation systems often face issues such as difficulty finding taxis, lack of transparency in pricing, and inefficient communication between passengers and drivers. Users need a digital platform that allows them to easily request rides and track trip information in real time.

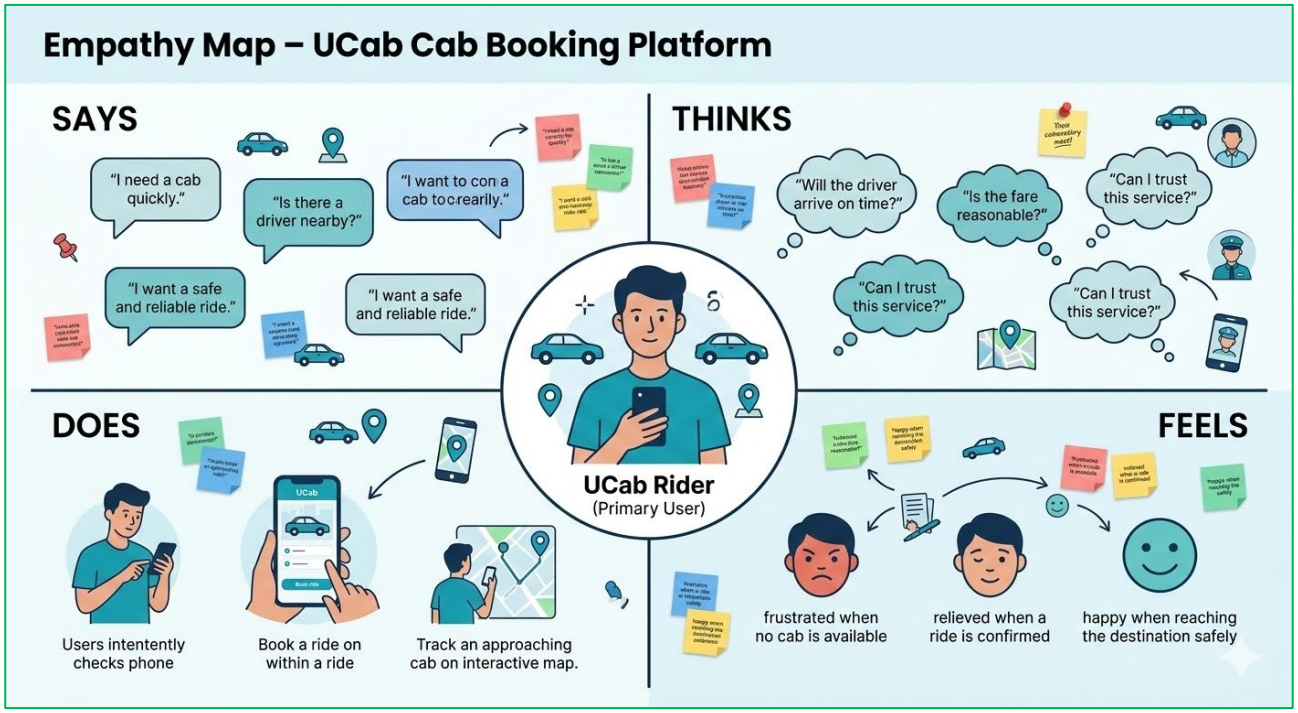
2.2 Empathy Map Canvas

Passenger Perspective:

- Needs quick transportation within the city
- Wants transparent ride information
- Wants a safe and reliable driver

Driver Perspective:

- Wants consistent ride requests
- Needs an easy way to manage rides
- Wants a platform to increase earnings



2.3 Brainstorming

Possible solutions discussed during brainstorming included:

- A mobile or web platform for ride booking
- Real-time ride request system
- Driver availability tracking
- Trip history and ride management
- Secure authentication and user management

The final solution selected was a **web-based cab booking system built with MERN stack**.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

1. User opens UCab application
2. User registers or logs into the system
3. Passenger enters pickup and drop locations
4. System searches for available drivers

5. Driver accepts the ride request
6. Ride begins and trip status is updated
7. Ride completes and trip history is stored

3.2 Solution Requirement

Functional Requirements:

- User registration and login
- Driver registration and login
- Ride request creation
- Driver ride acceptance
- Trip history management
- Secure authentication using JWT

Non-Functional Requirements:

- System should respond quickly
- Data should be stored securely
- The interface should be user-friendly
- System should handle multiple users simultaneously

3.3 Data Flow Diagram

The **Data Flow Diagram (DFD)** represents how data moves through the UCab Cab Booking Platform and how different entities interact with the system.

The diagram contains three main **external entities**: **Rider, Driver, and Admin**. These entities interact with the central system called **UCab Cab Booking System**. The rider initiates the ride booking process by sending a ride request to the system. The system processes the request and sends ride notifications to nearby drivers. The driver can accept or reject the ride request, and the response is sent back to the system.

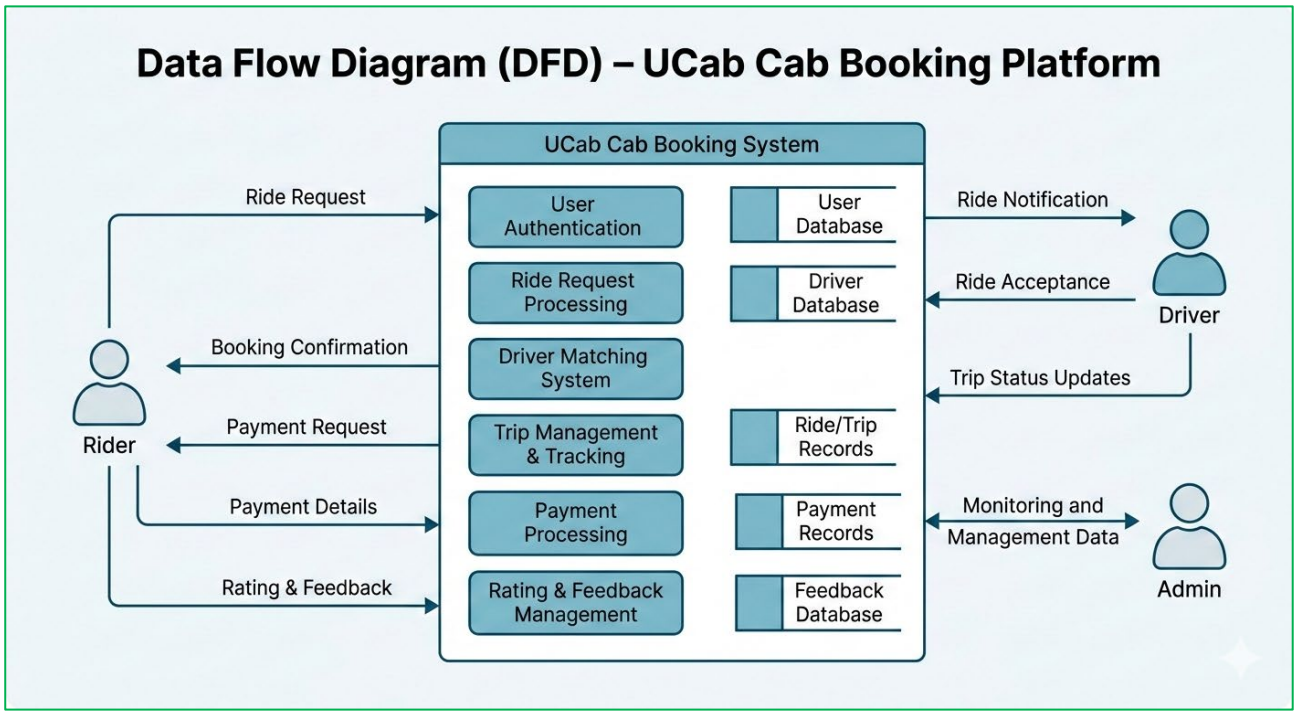
Once the ride is accepted, the system sends a **booking confirmation** to the rider. During the trip, the driver sends **trip status updates** such as ride start, ride progress, and ride completion. The system stores and processes this information in the database.

The system also manages **payment processing**, where payment requests are sent to the rider and payment details are stored in the payment records database. After the trip is completed, the rider can submit **ratings and feedback**, which are stored for service improvement.

The system uses multiple **data stores**, including the **User Database, Driver Database, Ride/Trip Records, Payment Records, and Feedback Database**. These databases maintain important information related to users, drivers, trips, payments, and feedback.

The **Admin entity** interacts with the system to monitor platform activity, manage users, and oversee ride operations.

Overall, the DFD illustrates the flow of information between users, system processes, and databases within the UCab platform.



3.4 Technology Stack

Frontend
- React.js

Backend
- Node.js
- Express.js

Database
- MongoDB

Authentication
- JWT (JSON Web Token)

Development Tools
- GitHub

- VS Code
- Node Package Manager (NPM)

4. PROJECT DESIGN

4.1 Problem Solution Fit

UCab addresses the problem of inefficient ride booking by providing a centralized platform where passengers can request rides and drivers can respond to ride requests in real time. The platform improves accessibility, convenience, and transparency.

4.2 Proposed Solution

The proposed system is a web-based ride booking platform where users can register, log in, request rides, and track trip history. Drivers can accept ride requests and manage their ride status through the system.

4.3 Solution Architecture

The **Solution Architecture** of the UCab Cab Booking Platform follows a **layered architecture model**, which separates the system into different functional layers to ensure scalability, maintainability, and efficient data flow.

At the **top layer**, the system includes three primary users: **Rider, Driver, and Admin**. Riders use the application to request rides and track trip details, drivers use the driver interface to receive and manage ride requests, and administrators use the admin dashboard to monitor system operations.

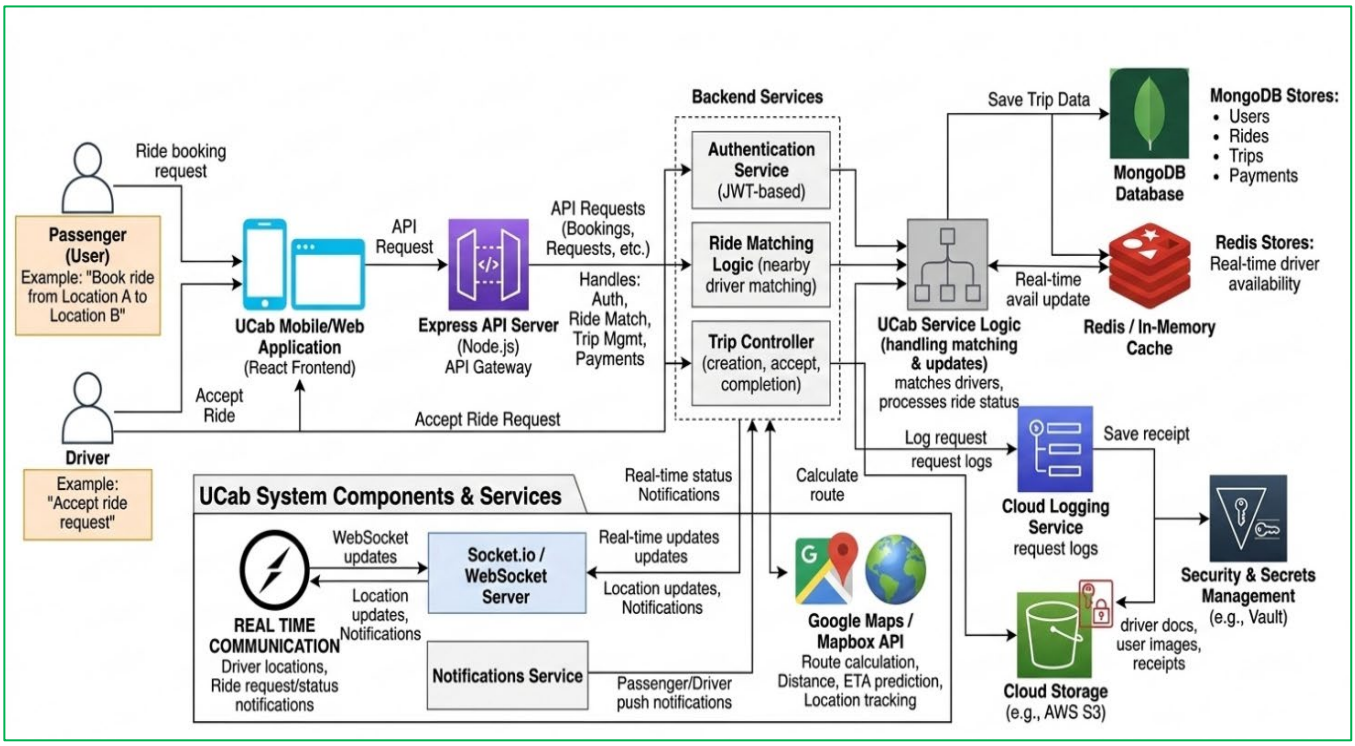
The **Application Interface Layer** acts as the user interaction layer. It consists of the **Rider App Interface**, **Driver App Interface**, and **Admin Dashboard**. The rider interface allows users to book rides, track trips, and make payments. The driver interface enables drivers to receive ride notifications, accept ride requests, and update trip status. The admin dashboard allows administrators to manage users and monitor system activities.

The **Backend Services Layer** forms the core logic of the system and is implemented using **Node.js and Express.js**. This layer includes modules such as **Authentication Service**, **Ride Request Processing**, **Driver Matching System**, **Trip Management**, **Payment Processing**, and **Feedback & Rating Management**. These services process requests from the application interface and handle all business logic.

The **Data Storage Layer** uses **MongoDB** to store application data. Separate databases are maintained for **User Data, Driver Data, Ride/Trip Records, Payment Data, and Feedback Data**. Backend services interact with these databases to store and retrieve system information.

The architecture follows a clear **data flow pattern** where requests move from **Users → Application Interface → Backend Services → Database**, and responses are returned back to users through the same path.

This layered architecture ensures efficient system communication, secure data management, and scalable service integration for the UCab platform.



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project follows the **Agile Scrum methodology**.

Sprints were divided into phases such as:

Sprint 1
User authentication and registration

Sprint 2
Ride booking and driver management

Sprint 3
Ride acceptance and trip management

Sprint 4
Testing, debugging, and performance optimization

Story points were used to estimate development effort for each task.

6. FUNCTIONAL AND PERFORMANCE TESTING

Functional testing was conducted to verify that all system features operate correctly.

Tested modules:

- User registration
- Login authentication
- Ride booking
- Driver ride acceptance
- Trip history

Test Case ID	Scenario (What to test)	Test Steps (How to test)	Expected Result	Actual Result	Pass/Fail
FT-01	User Registration Validation	Enter valid and invalid details in signup form (email, password, phone)	Valid data registers successfully, invalid inputs show validation errors	Registration worked correctly and validation messages displayed	Pass
FT-02	Login Authentication	Enter correct and incorrect login credentials	Correct credentials log user in, incorrect credentials show error	Login works correctly and invalid credentials rejected	Pass
FT-03	Ride Booking Functionality	Enter pickup and drop location and click “Book Ride”	Ride request created and driver search initiated	Ride request successfully created in system	Pass
FT-04	Driver Ride Acceptance	Driver logs in and accepts a ride request	Ride status changes from	Driver successfully	Pass

			"Requested" to "Accepted"	accepted ride and status updated	
FT-05	Real-time Location Tracking	Passenger tracks driver location during ride	Driver location updates in real time on map	Location updates correctly on map	Pass
FT-06	Trip History	User opens trip history section	Past rides are displayed correctly	Completed rides shown correctly	Pass

6.1 Performance Testing

Performance testing evaluated system response time and stability.

Tests performed:

- API response time test
- Concurrent ride request handling
- Database query performance

The system handled multiple requests efficiently with acceptable response times.

Test Case ID	**Scenario (What to test)**	**Test Steps (How to test)**	**Expected Result**	**Actual Result**	**Pass/Fail**
PT-01	Ride Request Response Time	Measure time taken after clicking "Book Ride"	Ride request processed within 2–3 seconds	Ride created within acceptable time	Pass
PT-02	Concurrent User Load	Simulate multiple users requesting rides simultaneously	System handles requests without crashes	System remained stable under multiple requests	Pass
PT-03	Real-time Updates Performance	Track driver location while ride is active	Location updates should appear instantly	Real-time updates work smoothly	Pass

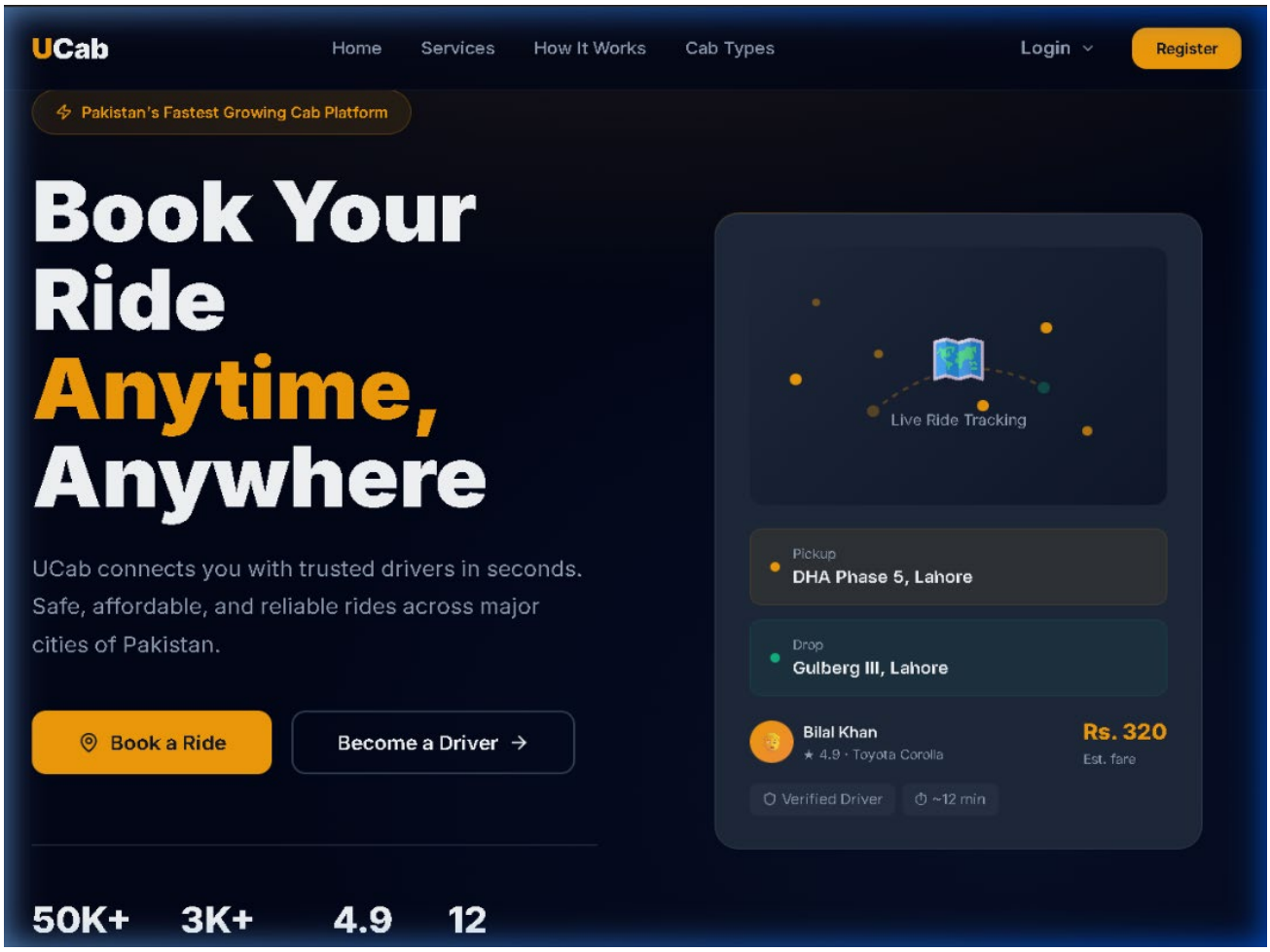
PT-04	Database Query Performance	Fetch trip history from database	Data retrieved quickly without delay	Query executed successfully	Pass

7. RESULTS

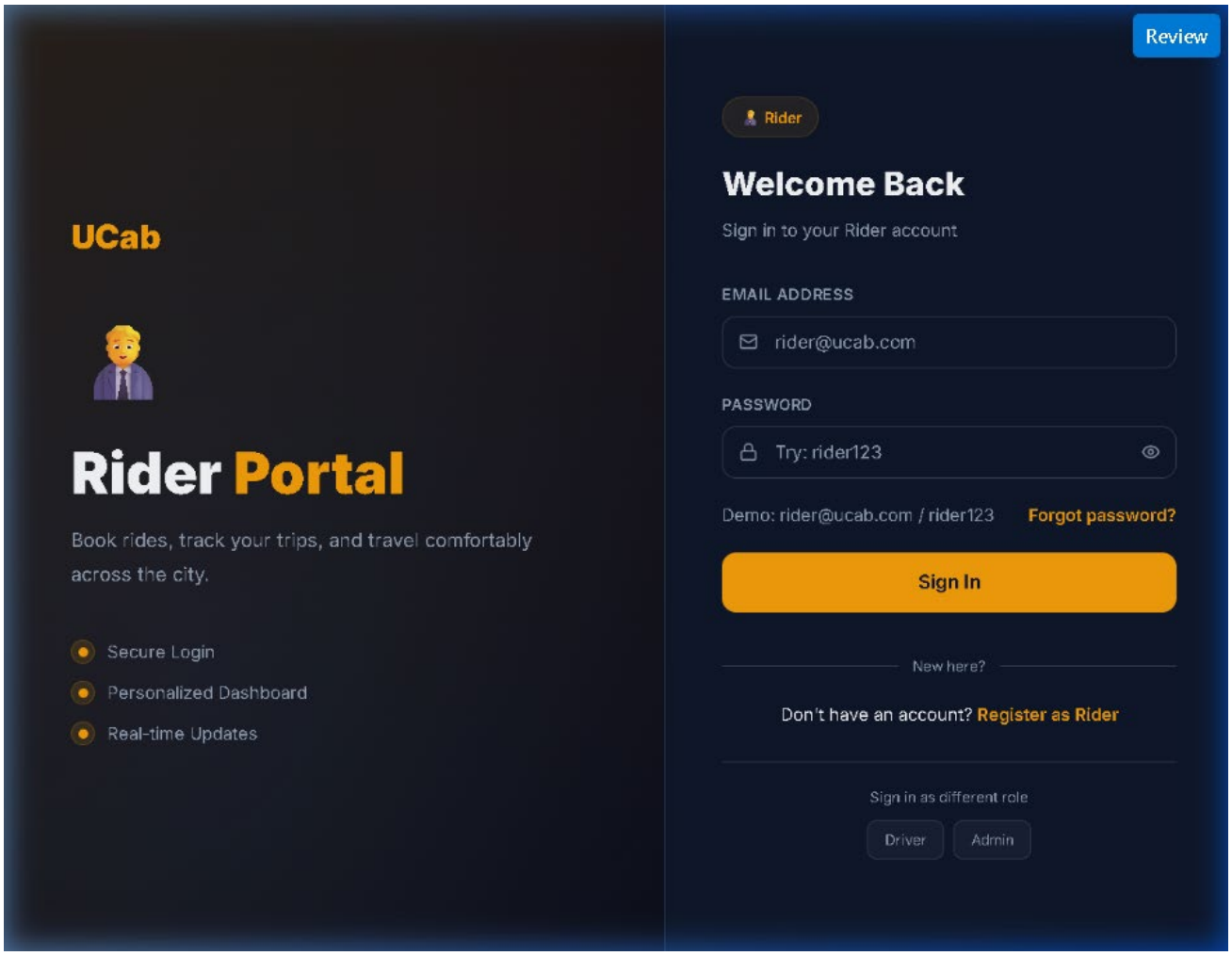
7.1 Output Screenshots

Screenshots of the project modules:

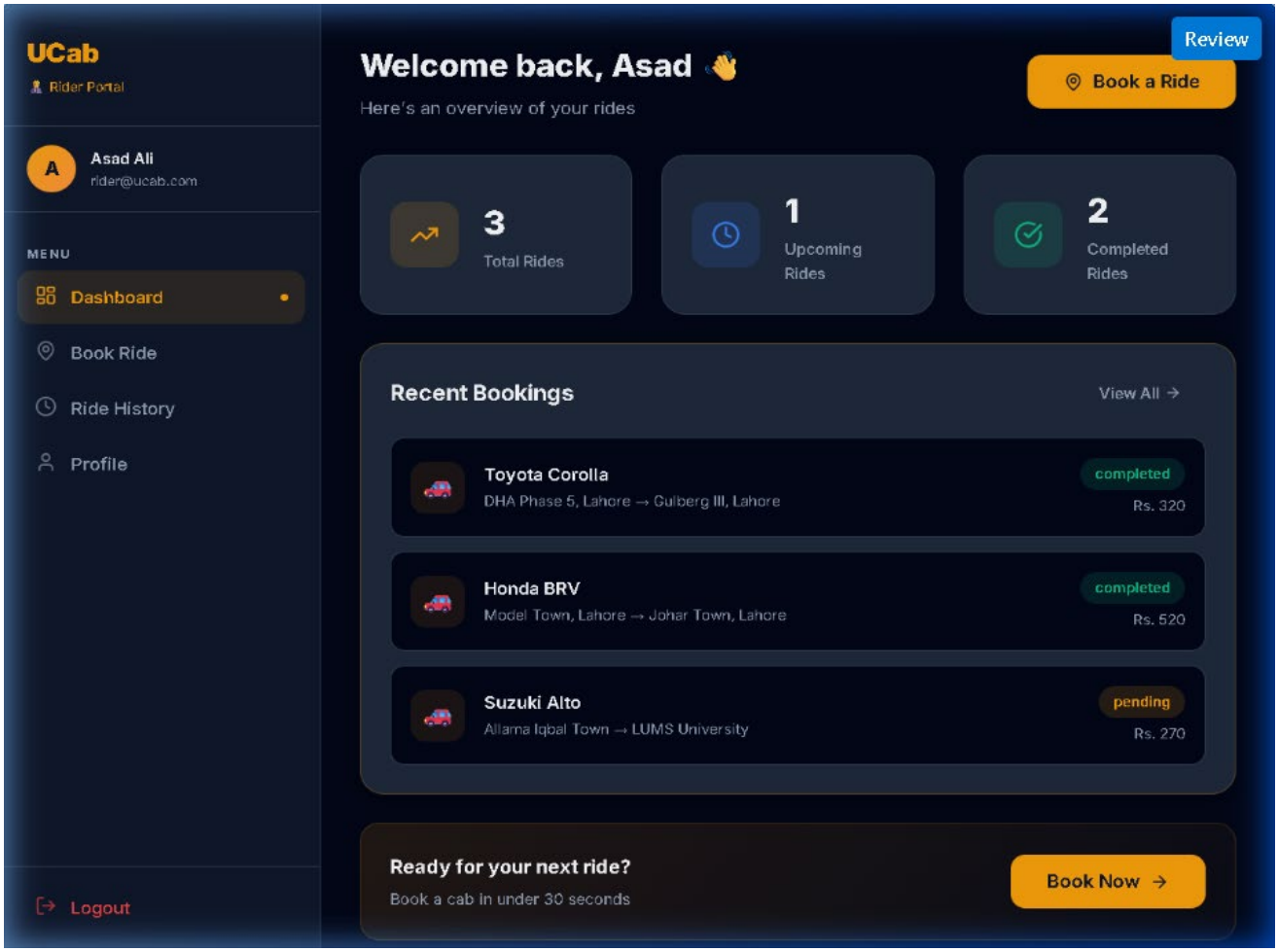
- Home Page



- User registration page



- User Dashboard



8. ADVANTAGES & DISADVANTAGES

Advantages:

- Easy ride booking process
- Centralized platform for passengers and drivers
- Secure authentication
- Scalable architecture using MERN stack

Disadvantages:

- Requires internet connectivity
- Real-time driver tracking may require additional APIs

- Limited to basic ride booking features

9. CONCLUSION

The UCab project successfully demonstrates the development of a cab booking platform using the MERN stack. The system provides a functional interface for passengers to request rides and for drivers to manage ride requests. The project highlights the effectiveness of modern web technologies in developing scalable transportation services.

10. FUTURE SCOPE

Future improvements may include:

- Mobile application development
- Real-time GPS tracking using map APIs
- Online payment integration
- Driver rating and review system
- Push notification support
- Advanced ride matching algorithms

11. APPENDIX

Source Code:
Available on GitHub.

Dataset Link:
Not applicable (system uses user-generated ride data).

GitHub & Project Demo Link:
https://github.com/itsasad786/Ucab

Submitted By:

1. Asad Ali (Team Lead)
2. Ayushi Gupta
3. Mahak Vishwakarma
4. Ganguli Kumar Yadav